LLM Quantization and Evaluation

CS290W LLM15 Lecture

LU Yuzhou

Lab Overview

Today's Topics:

1. **Quantization Fundamentals**: Outliers, sensitivity analysis
2. **Modern Methods**: GPTQ, AWQ, GGUF/K-quants
3. **Compression**: Pruning with Wanda
4. **Evaluation**: Perplexity, MMLU, GSM8K, hardware metrics

Why This Matters

Quantization reduces model size by 4-8× while maintaining performance, enabling LLM deployment on edge devices and consumer hardware.

- **Recommended Hardware**: AMD Ryzen AI Halo Processors or NVIDIA GPU with 8GB+ VRAM
- **Environment**: ROCm + PyTorch via AUP Learning Cloud

What is Quantization?

Definition: Converting model weights and activations from high-precision to lower-precision formats.

Precision Formats

- **FP32/BF16** $\rightarrow$ **INT8/INT4** (weights)
- **FP16** $\rightarrow$ **INT8** (activations)

Benefits:

- **Memory**: 4$\times$ smaller model (FP32 $\rightarrow$ INT8)
- **Bandwidth**: Reduced data transfer
- **Speed**: Faster inference on edge devices

Example: 1B Parameter Model

FP32: 4 GB $\rightarrow$ INT8: 1 GB $\rightarrow$ INT4: 0.5 GB

Quantization Fundamentals

Uniform Quantization Formula:

Symmetric Quantization

$$\text{scale} = \frac{\max(|x|)}{q_{\max}}$$

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right), \text{clipped to } [q_{\min}, q_{\max}]$$

$$x_{\text{dequant}} = x_q \cdot \text{scale}$$

Where:

- $q_{\min} = -(2^{\text{bits}-1})$ (e.g., -128 for INT8)
- $q_{\max} = 2^{\text{bits}-1} - 1$ (e.g., 127 for INT8)

Quantization Error (MSE):

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2$$

Why Outliers Matter

Outliers are extreme values in weights and activations that emerge when models exceed $\sim$6.7B parameters.

Outlier Definition (LLM.int8())

A feature dimension h_i is an outlier if:

1. Absolute value ≥ 6
2. Appears in $\geq 25\%$ of transformer layers
3. Appears in $\geq 6\%$ of sequence dimensions

Impact on Quantization:

- Increase quantization step size Δ, reducing precision for normal values
- Concentrate in specific feature dimensions
- Critical for contextual knowledge and reasoning

Outliers Increase Quantization Error

Key Insight: Outliers expand the quantization range, reducing precision for normal values.

Tensor	Range	Bits	MSE	SNR
Normal	$[-3.4, 3.4]$	8-bit	0.000062	42.16dB
Normal	$[-3.4, 3.4]$	4-bit	0.018200	17.50dB
With Outliers	$[-12.1, 16.1]$	8-bit	0.001024	33.15dB
With Outliers	$[-12.1, 16.1]$	4-bit	0.284479	8.71dB

Key Insight

Outliers increase the quantization scale, reducing precision for normal values. 4-bit quantization is especially affected.

Layer-wise Sensitivity Analysis

Different layers exhibit varying sensitivity to quantization:

Layer Type	Sensitivity	Recommendation
Input/Embedding	High	Keep in FP16/BF16
Attention Output	High	Keep in FP16/BF16
FFN Gate/Down	Medium-High	Consider mixed precision
Bottleneck Layers	Low	Can quantize to INT4
Output Layers	Moderate	INT8 recommended

Mixed-Precision Strategy

- Keep sensitive layers in higher precision (FP16/BF16)
- Aggressively quantize non-sensitive layers (INT4)
- Optimize memory without significant accuracy loss

Modern LLM Quantization Methods

Method	Type	Precision	Key Innovation
GPTQ	PTQ (Weight-Only)	W3A16, W4A16	Hessian-based compensation
AWQ	PTQ (Weight-Only)	W4A16	Activation-aware protection
SmoothQuant	PTQ (W&A)	W8A8	Migrate outliers to weights
LLM.int8()	PTQ (W&A)	W8A8	Mixed-precision for outliers
GGUF	PTQ	Q2_K to Q8_0	K-quants for fine control

PTQ vs QAT

- **PTQ (Post-Training Quantization)**: After training, using calibration data (no retraining)
- **QAT (Quantization-Aware Training)**: Simulate quantization during training (higher accuracy but expensive)

GPTQ: Hessian-Based Compensation

GPTQ uses second-order information to minimize quantization error.

Key Equation:

$$\min_q \sum_j H_{ij}(w_j - q_j)^2$$

where H is the Hessian matrix of the loss function.

Algorithm:

1. Process weights column-by-column (or in blocks)
2. Quantize weight w_i to nearest value q_i
3. Update remaining weights: $w_j \leftarrow w_j - \frac{H_{ij}}{H_{ii}}(w_i - q_i)$
4. Use Cholesky decomposition for stable Hessian inverse

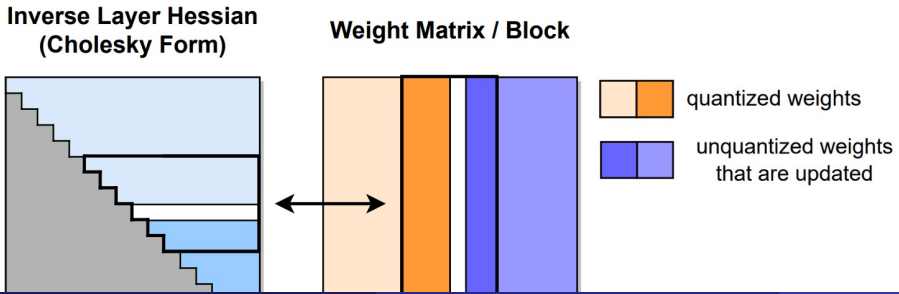
Optimizations

- **Fixed Order**: Predetermined quantization order
- **Lazy Batch Updates**: Delay updates to batches (e.g., 128 columns)
- **Group Size**: Quantize in groups (e.g., 128) for better tradeoff

GPTQ: Error Reduction

GPTQ reduces quantization error by compensating with Hessian information.

Method	4-bit MSE
Simple Quantization	0.026443
GPTQ	0.013969
Improvement	**47.17%**



AWQ: Activation-Aware Weight Quantization

Key Insight: Not all weights are equally important. A small fraction (0.1%-1%) of "salient weights" dominates model performance.

Salient Weight Identification:

$$\mathcal{S} = \{(i, j) : |\mathcal{X}_j| > \tau\}$$

where $\mathcal{X}_j$ is the activation of channel j.

Protection Strategy:

$$\mathbf{W}' = \mathbf{W} \cdot \text{diag}(\mathbf{s})$$
$$\mathbf{X}' = \mathbf{X} \cdot \text{diag}(\mathbf{s})^{-1}$$

Key Point

This keeps the output mathematically equivalent while reducing quantization error for important weights.

AWQ: Error Reduction

AWQ protects important weights by scaling based on activation magnitude.

Method	4-bit MSE
Simple Quantization	0.052326
AWQ	0.036199
Improvement	**30.82%**

TEXT: Insert diagram showing salient channel identification and scaling

Key Insight

Channels with high activation are scaled to reduce quantization error. AWQ generally outperforms GPTQ in accuracy retention.

GGUF and K-Quants

GGUF (GPT-Generated Unified Format) is the quantization format used by llama.cpp with **K-quants** for fine-grained control.

Type	Size (7B)	Quality
Q2_K	∼2.8 GB	Low
Q3_K_M	∼3.8 GB	Medium
Q4_K_S	∼4.4 GB	Good
Q4_K_M	∼4.7 GB	Very Good
Q5_K_M	∼5.5 GB	Excellent
Q6_K	∼6.0 GB	Near-lossless
Q8_0	∼7.0 GB	Virtually lossless

K-Quant Nomenclature

- **Q4_K_S (Small)**: 4-bit uniform
- **Q4_K_M (Medium)**: 6-bit for critical tensors (gate_proj, down_proj)

Why Mixed Precision in K-Quants?

Similar to sensitivity analysis, K-quants recognize that some tensors are more important:

- **Attention output tensors**: Critical for context understanding
- **FFN gate/down projections**: Important for reasoning
- **Embedding layers**: Sensitive to quantization

Example: Q4_K_M

Uses 6-bit for critical tensors while keeping 4-bit for others, achieving better quality with minimal size increase.

Key Features:

- Per-tensor quantization with different bit-widths
- Optimized for CPU inference
- Supports partial GPU offloading
- Efficient memory mapping for fast loading

LLM Compression and Pruning

Pruning removes redundant parameters to reduce model size and computation.

Type	Method	Result
Unstructured	Remove individual weights	Sparse matrices
Structured	Remove rows/columns/heads	Dense smaller matrices

Wanda: Pruning by Weights and Activations

Importance Score:

$$\mathcal{I}_{i,j} = |\mathbf{W}_{i,j}| \cdot \|\mathbf{X}_j\|_2$$

Advantages

- No training required (PTQ-compatible)
- Works well with quantization
- Preserves model performance better than magnitude-only pruning

Wanda Pruning Demo

Algorithm:

1. Compute importance scores: $|W| \cdot \|X\|_2$
2. For each output neuron, prune weights with lowest importance
3. Maintain sparsity ratio (e.g., 50% pruning)

Example Results

- Sparsity 30%: Output MSE = 3.80
- Sparsity 50%: Output MSE = 18.67
- Sparsity 70%: Output MSE = 57.81

Combined Approach:

1. Apply Wanda pruning (e.g., 50% sparsity)
2. Quantize remaining weights to INT4 using AWQ/GPTQ
3. Result: $\sim 8\times$ compression ($2\times$ from pruning, $4\times$ from quantization)

Evaluation Methodologies

Three Categories of Metrics:

1. **Intrinsic Metrics**: Perplexity (PPL)
2. **Extrinsic Benchmarks**: MMLU, GSM8K, ARC, HellaSwag
3. **Hardware Metrics**: TPS, Latency, VRAM

Perplexity Formula

$$\text{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i|w_{1:i-1})\right)$$

- Lower perplexity = better language modeling
- Typical: 10-20 (excellent), 20-50 (good), 50+ (needs improvement)
- Quantized models should have $<5\%$ PPL increase vs. baseline

Perplexity Interpretation

PPL Range	Quality
< 10	Exceptional (human-level)
10-20	Excellent
20-50	Good
50-100	Fair
> 100	Poor

Quantization Impact on PPL

- 4-bit quantization: +2-5% PPL increase
- 8-bit quantization: <1% PPL increase
- AWQ/GPTQ preserve PPL better than naive quantization

MMLU Benchmark

MMLU (Massive Multitask Language Understanding):

- 57 tasks across STEM, humanities, and professional domains
- 15,908 multiple-choice questions
- Measures knowledge and reasoning
- Score: Accuracy percentage (0-100%)

Expected Scores for Qwen3-0.6B

- BF16 (baseline): $\sim$35-45%
- 4-bit AWQ: $\sim$33-43% (2-3% drop)
- 4-bit GPTQ: $\sim$32-42% (3-4% drop)
- 8-bit: $\sim$34-44% ($<$1% drop)

For comparison, larger models (4B+) score 65-80%

GSM8K and Other Benchmarks

GSM8K (Grade School Math 8K):

- 8,500 grade school math problems
- 2-8 reasoning steps per problem
- Measures mathematical reasoning
- Score: Accuracy percentage (0-100%)

Other Important Benchmarks:

- **ARC**: Science questions (Easy + Challenge)
- **HellaSwag**: Commonsense reasoning
- **TruthfulQA**: Hallucination resistance
- **Winogrande**: Pronoun resolution

TEXT: Insert chart comparing benchmark scores across quantization methods

Hardware Performance Metrics

Key Metrics:

1. **Token Throughput (TPS)**: Tokens generated per second
   - Higher is better; Typical: 20-100 tokens/s
2. **Latency**: Time to first token + time per token
   - Prefill latency: Process input prompt
   - Decode latency: Per generated token
3. **VRAM Occupancy**: GPU memory usage

VRAM Calculation:

$$\text{VRAM} = \text{Model Size} + \text{KV Cache} + \text{Overhead}$$

$$\text{Model Size} = \text{Parameters} \times \text{Bytes per parameter}$$

Performance Comparison

Metric	BF16	4-bit AWQ	4-bit GPTQ	GGUF Q4_K_M
VRAM	∼8 GB	∼3 GB	∼3 GB	∼3 GB
TPS	50-80	80-120	70-100	30-50 (CPU)
PPL	baseline	+2-3%	+3-5%	+3-4%
MMLU	baseline	-2-3%	-3-4%	-3-4%

Key Takeaways

- 4-bit quantization reduces model size by 60-70% with minimal quality loss
- AWQ generally provides better accuracy than GPTQ at 4-bit
- GGUF Q4_K_M offers excellent CPU inference performance
- Mixed-precision (K-quants) preserves quality for sensitive tensors

Summary: Key Takeaways

1. **Quantization Fundamentals**: Outliers significantly impact low-bit quantization; sensitivity analysis guides mixed-precision strategies
2. **Modern Methods**:
   - GPTQ: Hessian-based error compensation (47% error reduction)
   - AWQ: Activation-aware weight protection (31% error reduction)
   - GGUF: K-quants for fine-grained CPU inference
3. **Pruning**: Wanda combines weights and activations for effective sparsity
4. **Evaluation**: Use perplexity, MMLU, GSM8K, and hardware metrics together
5. **Best Practices**:
   - 4-bit AWQ for GPU inference (best accuracy/size tradeoff)
   - GGUF Q4_K_M for CPU inference
   - Always evaluate on your target tasks

Experiment Further

- Quantize Qwen3-0.6B using AutoGPTQ, AutoAWQ, and llama.cpp
- Compare 4-bit vs 8-bit quantization on your downstream tasks
- Apply Wanda pruning before quantization for additional compression
- Benchmark TPS and VRAM on your hardware
- Evaluate on MMLU subsets relevant to your use case

Resources

- **GPTQ Paper**: arXiv:2210.17323
- **AWQ Paper**: arXiv:2306.00978
- **Wanda Paper**: arXiv:2306.11695
- **llama.cpp**: GitHub
- **Lab Notebook**: `LLM15-Quantization-and-Evaluation.ipynb`